

6CS012

Artificial Intelligence and Machine Learning

Portfolio Task 1: Visual Task

Full Name	:	Narayan Lohani
Student Number	:	2408869
Course	:	BSc (Hons) Computer Science
University Email	:	n.lohani@wlv.ac.uk
Group	:	L6CG15
Tutor	:	Jinu Nyachhyon
Module Leader	:	Siman Giri
Submission Date	:	May 10, 2026

GitHub Visual Task Folder Link: [Visual-Task-Folder](#)

GitHub Repo Link: [6CS012 - Narayan Lohani](#)

Table of Contents

1. Title	1
2. Abstract	1
3. Introduction	2
4. Dataset	3
4.1. Source and Description:	3
4.2. Class Selection	3
4.3. Class Distribution	4
4.4. Preprocessing	5
5. Methodology	7
5.1. Baseline Model	7
5.2. Deep CNN with Regularisation	9
6. Experiments and Results	11
6.1. Baseline vs Deeper Architecture	11
6.2. Computational Efficiency	12
6.3. Training with Different Optimizers	13
6.4. Ablation Study: Removing Dropout	14
6.5. Challenges in Training	15
7. Transfer Learning	16
7.1. ResNet50 Transfer Learning Model	17
8. Conclusion and Future Work	20
9. References	21

Table of Figures

Figure 1: Visualizing one sample from each fruit class.....	4
Figure 2: Class distribution of training set.....	4
Figure 3: Class distribution of test set.....	5
Figure 4: Visualize an augmented sample from each sample	6
Figure 5: Training set class distribution after Augmentation	6
Figure 6: Baseline Model: train vs validation accuracy and loss	8
Figure 7: Baseline Model Confusion Matrix	8
Figure 8: Deep Model: train vs validation accuracy and loss	10
Figure 9: Deep Model Confusion Matrix.....	10
Figure 10: Baseline vs Model: loss and accuracy comparision	11
Figure 11: Accuracy and Loss plot of SGD vs Adam optimizer in deeper model	13
Figure 12: Accuracy and Loss plot of deeper model with and without dropout.....	14
Figure 13: ResNet50 Model: train vs validation accuracy and loss.....	18
Figure 14: ResNet50 Model Confusion Matrix	18

Table of Tables

Table 1: List of Selected Fruits	3
Table 2: Baseline Model Architecture	7
Table 3: Deep Model Architecture	9
Table 4: Baseline vs Deep model Performance Comparision	11
Table 5: Basline vs Deep Model Computational Efficiency Comparision	12
Table 6: Adam vs SGD comparision in Deep model.....	13
Table 7: Comparison of Model with and without Dropout	14
Table 8: Resnet Model Architecture	17
Table 9: Deep model vs ResNet50 Performance	19

Title

A Performance Analysis of Deep Learning Architectures for the Classification of 15 Selected Fruit Varieties using CNNs and Transfer Learning

Abstract

This report contains application and analysis of Convolutional Neural Networks (CNNs) for multi-class fruit image classification using a curated 15-class subset of the Fruits-360 dataset. The study is structured around two primary components: (A) training CNNs entirely from scratch, and (B) leveraging transfer learning via a pre-trained ResNet50 model.

In Part A, a Baseline CNN (3 convolutional blocks, 3 fully connected layers) was designed and trained using the SGD optimizer, achieving a competitive test accuracy. A deeper regularized architecture (6 convolutional blocks augmented with Batch Normalization, Dropout, L2 weight decay, and GlobalAveragePooling) was subsequently developed and evaluated under both SGD and Adam optimizers. An ablation study was conducted to quantify the individual contribution of Dropout to generalization performance.

In Part B, a pre-trained model on ImageNet ResNet50 was used to classify the same dataset and compare its performance metrics with CNN from scratch.

Key findings indicate that deeper architectures with regularization substantially outperform the baseline. Adam converges significantly faster than SGD with momentum and Dropout is a critical component for controlling the train/validation accuracy gap. ResNet50 transfer learning yields high accuracy ($\geq 99.5\%$) with markedly faster convergence than training from scratch. All experiments were conducted on Kaggle's GPU environment (T4 x2), with training times ranging from approximately 54 seconds for the Baseline CNN to over 260 seconds for the deeper architectures.

Introduction

Fruit recognition is a practically significant problem in domains ranging from agriculture to dietary monitoring and retail checkout systems. The ability to automatically classify fruit species from images eliminates the need of manual inspection, reduces human error, and enables real-time embedded deployment at scale.

Deep learning, and Convolutional Neural Networks (CNNs) in particular, have established themselves as the dominant paradigm for image classification since the ILSVRC2012 (ImageNet Large Scale Visual Recognition Challenge) (Krizhevsky, et al., 2017). Unlike hand-crafted feature pipelines, CNNs learn hierarchical spatial representations directly from pixel data, progressing from low-level edge detectors in beginning layers to complex patterns in deeper layers making them suitable for the visual diversity in natural fruit images.

The challenges addressed in this work is designing and comparing CNN architectures that generalize well to unseen fruit images, given a moderate-sized dataset. Specifically, this project investigates:

- Whether increased architectural depth and regularization (Batch Normalization, Dropout, L2 decay) meaningfully improve generalization over a shallow baseline.
- How the choice of optimizer (SGD with Nesterov momentum versus Adam) affects convergence speed and peak accuracy.
- Whether pre-trained ImageNet features (ResNet50) transfer effectively to fruit classification, and how this compares to training from scratch.
- The contribution of Dropout layers to preventing overfitting,

Dataset

Source and Description:

The dataset used in this project is a curated 15-class subset of the publicly available Fruits-360 dataset. (Oltean, 2017). Fruits-360 contains images of fruits, vegetables, nuts, and seeds captured by rotating specimens on the shaft of a low-speed motor (~ 3 rpm) and photographing them against a uniform white background. This controlled acquisition protocol produces visually clean, consistently lit images that are free of complex scene backgrounds, making the classification signal strong but still representative of color, texture, and shape-based discrimination challenges.

All images in Fruits-360 are 100×100 pixels in RGB format. The original dataset was accessed from the Kaggle which maintains the Training and Test subdirectory split.

Class Selection

The following 15 fruit classes were programmatically extracted from the full Fruits-360 collection:

Index	Original Folder Name	Normalised Label
1	Apple 8	apple
2	Banana Lady Finger 1	banana
3	Blueberry 1	blueberry
4	Carambola 1	carambola
5	Dates 1	dates
6	Fig 1	fig
7	Guava 1	guava
8	Kiwi 1	kiwi
9	Mango 1	mango
10	Orange 1	orange
11	Pepper 2	pepper
12	Pineapple 1	pineapple
13	Rambutan 1	rambutan
14	Strawberry 1	strawberry
15	Watermelon 1	watermelon

Table 1: List of Selected Fruits

Folder names were normalized to their base fruit name in lowercase (e.g., "Banana Lady Finger 1" → "banana") using a custom `rename_fruit_class_name()` function, providing clean, consistent class labels for downstream processing.

Visualization:

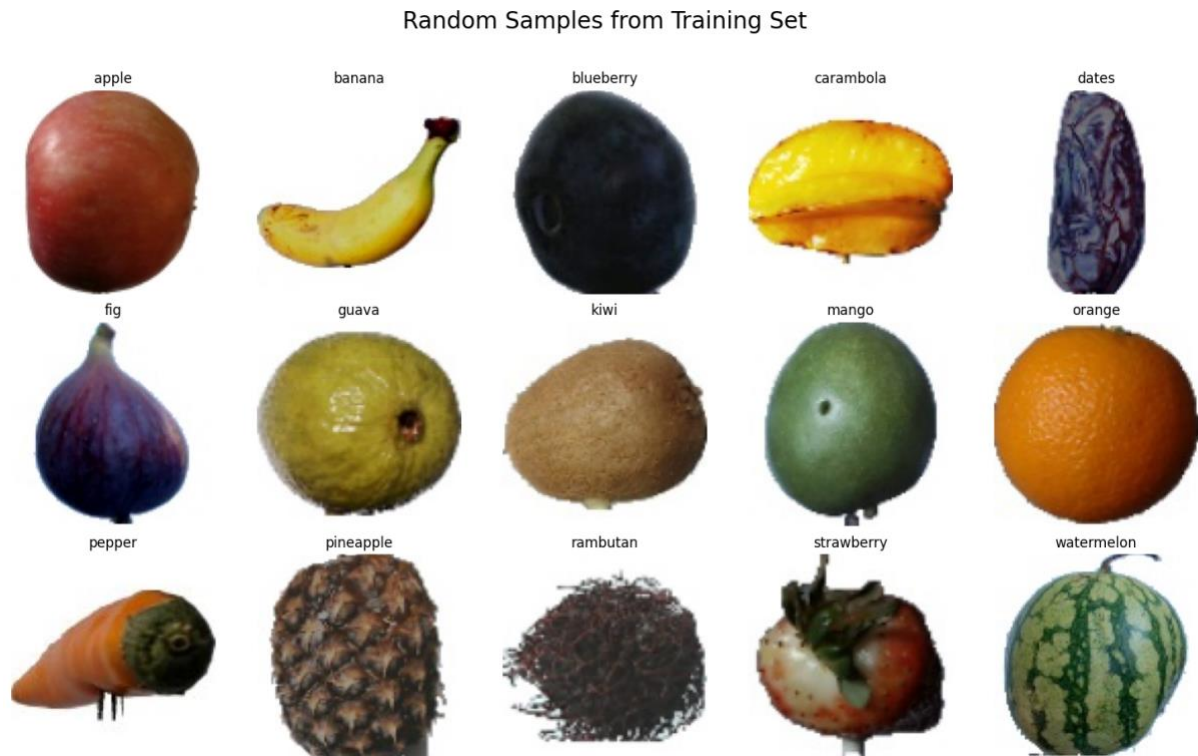


Figure 1: Visualizing one sample from each fruit class

Class Distribution

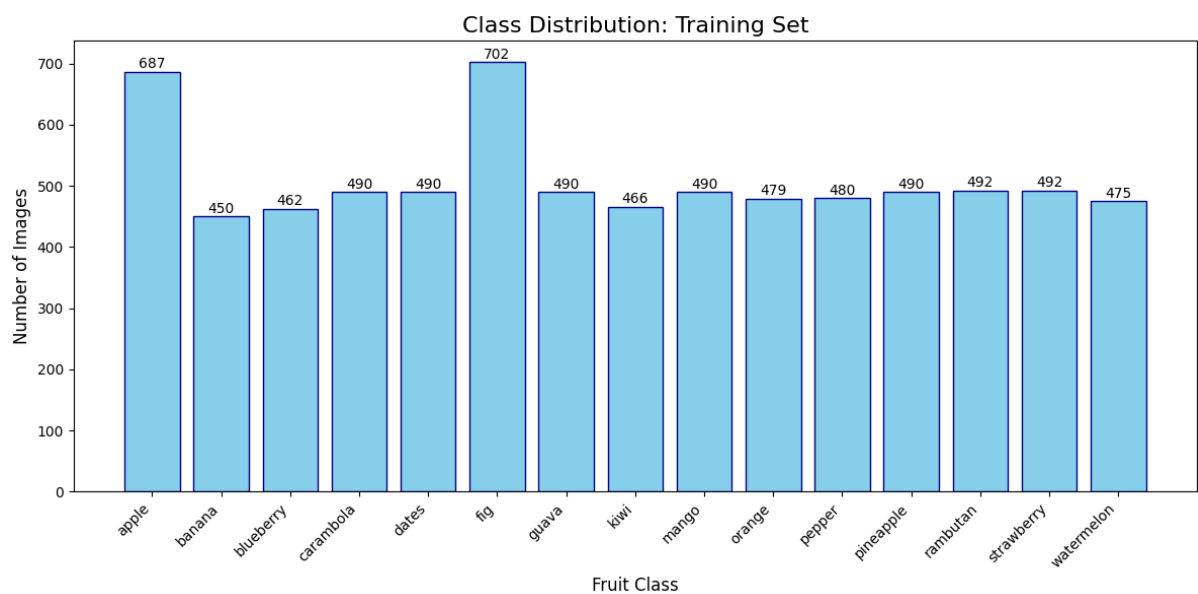


Figure 2: Class distribution of training set

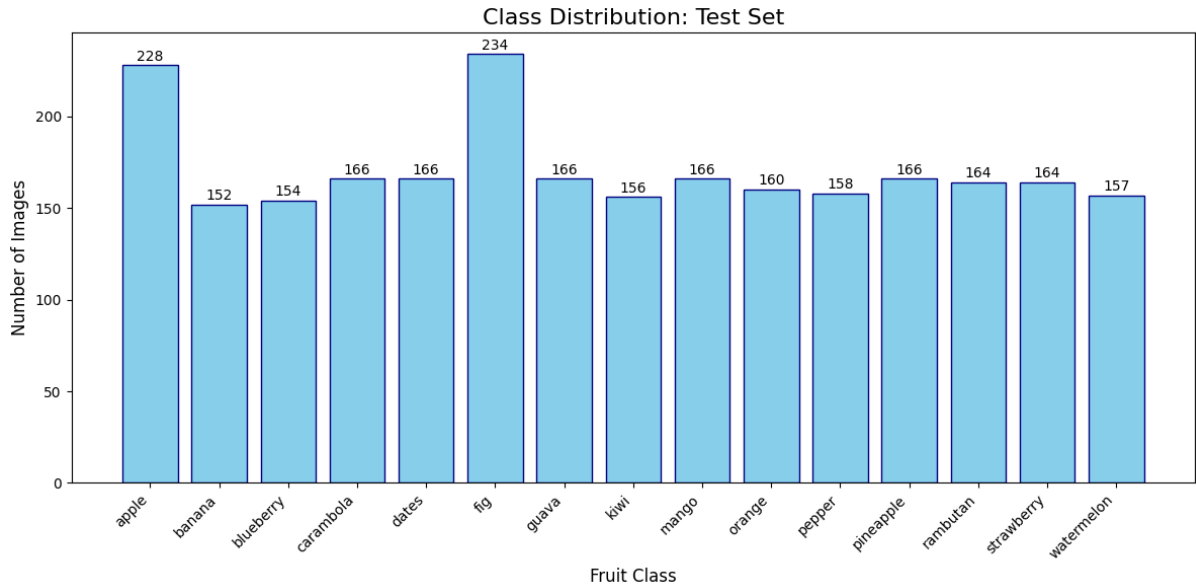


Figure 3: Class distribution of test set

Preprocessing

The preprocessing pipeline comprised three stages:

Image Resizing

For Part A (CNN from scratch): images were resized from their native 100×100 px to **128×128 pixels** using nearest-neighbour interpolation to preserve sharp edges.

For Part B (ResNet50): images were resized to **224×224 pixels** to match ResNet50's expected input resolution.

Normalisation

For scratch-trained models, pixel values were rescaled from $[0, 255]$ to $[0, 1]$ via a Keras Rescaling($1.0/255$) layer embedded directly in the model graph.

For the ResNet50 model, the `keras.applications.resnet50.preprocess_input` function was applied, which performs channel-wise mean subtraction (ImageNet statistics) and converts from RGB to BGR format, as required by the original ResNet50 training configuration.

Image Augmentation

A multi-strategy augmentation pipeline was applied to the training set only to artificially expand the dataset to a TARGET_SIZE of 1,000 images per class. The pipeline was implemented using TensorFlow, OpenCV (cv2), and PIL, applying following transformations per image from the following pool:

1. Horizontal flip (`tf.image.flip_left_right`)
2. Random rotation: $\pm 30^\circ$ (`cv2.warpAffine`, white background fill)
3. Brightness adjustment: $\pm 30\%$ delta (`tf.image.adjust_brightness`)
4. Contrast scaling: factor $\in [0.70, 1.30]$ (`tf.image.adjust_contrast`)
5. Zoom / central crop: fraction $\in [0.70, 1.00]$, resized back to original
6. Translation: up to 7% of image dimension (`cv2.warpAffine`, reflective border)

7. Gaussian noise: $\text{std} \in [5.0, 25.0]$ pixel units
8. Gaussian blur: $\text{kernel} \in \{3, 5, 7\}$, $\text{sigma} \in [0.5, 2.0]$

Augmented Samples

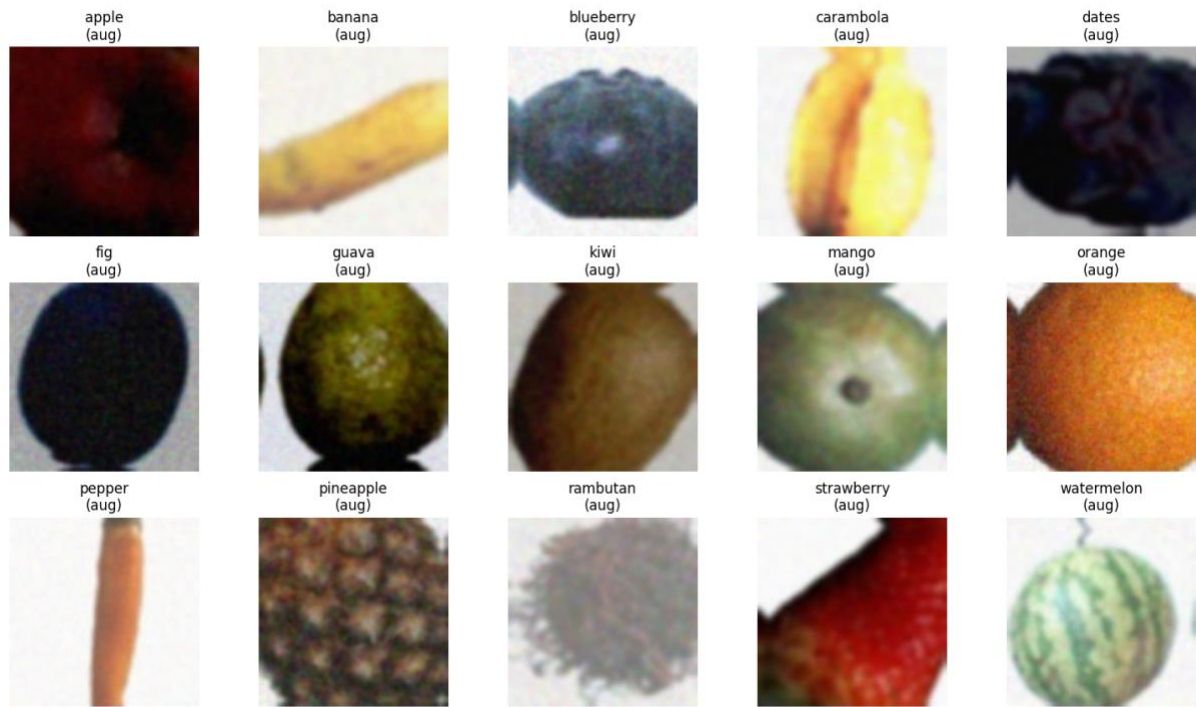


Figure 4: Visualize an augmented sample from each sample

Augmented images were saved as JPEG (quality=95) with the naming convention `aug_<class>_<index>.jpg`. A corruption-checking pass (`PIL img.verify()`) was subsequently applied to remove any malformed images. Lightweight online augmentation (`RandomFlip`, `RandomRotation`, `RandomZoom`, `RandomTranslation`) was additionally embedded in the scratch model graph to provide runtime variation during training.

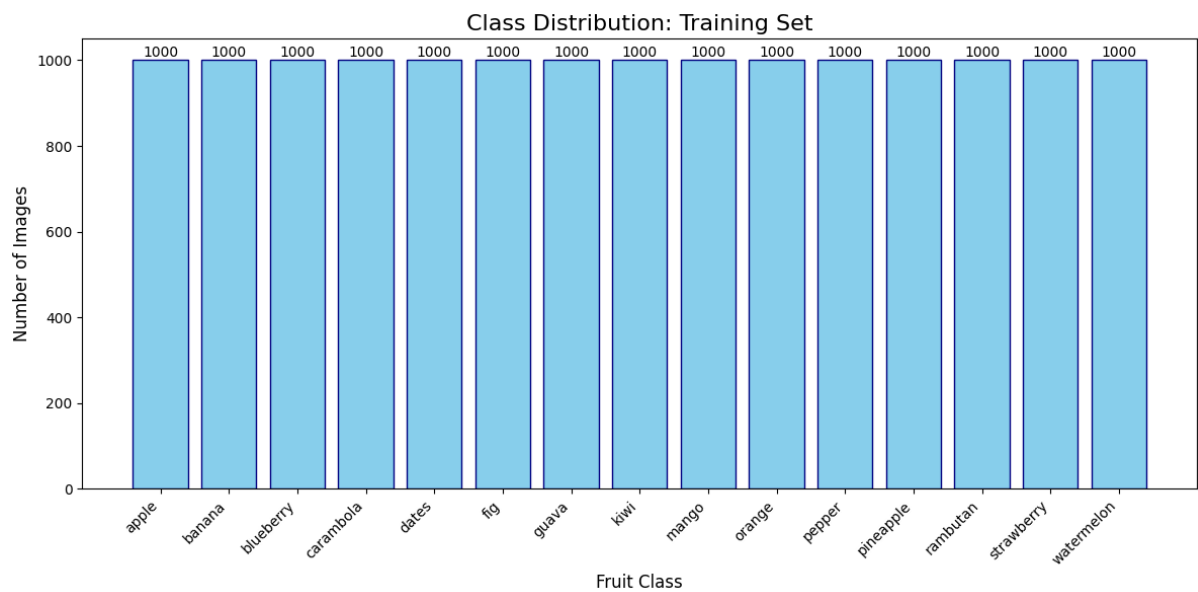


Figure 5: Training set class distribution after Augmentation

Methodology

Baseline Model

The Baseline CNN was designed as a minimal but functional convolutional network to establish a performance lower bound against which deeper architectures could be compared. Its architecture is given below:

Layer Type	Configuration	Output Shape
Input	$128 \times 128 \times 3$	$128 \times 128 \times 3$
Online Augmentation	RandomFlip, RandomRotation(0.3), RandomZoom(0.3), RandomTranslation(0.2)	$128 \times 128 \times 3$
Rescaling	1/255	$128 \times 128 \times 3$
Conv2D + MaxPool	16 filters, 3×3, ReLU, same padding; Pool 2×2	$64 \times 64 \times 16$
Conv2D + MaxPool	32 filters, 3×3, ReLU, same padding; Pool 2×2	$32 \times 32 \times 32$
Conv2D + MaxPool	64 filters, 3×3, ReLU, same padding; Pool 2×2	$16 \times 16 \times 64$
Flatten	—	16,384
Dense	64 units, ReLU	64
Dense	32 units, ReLU	32
Dense	16 units, ReLU	16
Dense (Output)	15 units, Softmax	15

Table 2: Baseline Model Architecture

Optimiser: SGD (learning_rate = 0.005)

Loss: Sparse Categorical Cross-Entropy

Batch size: 32

Max Epochs: 10, with EarlyStopping (patience=5) monitoring val_loss

Callbacks: ModelCheckpoint on validation accuracy, EarlyStopping on validation loss with patience of 5



Figure 6: Basline Model: train vs validation accuracy and loss

Test Loss : 0.2877

Test Accuracy : 0.9194

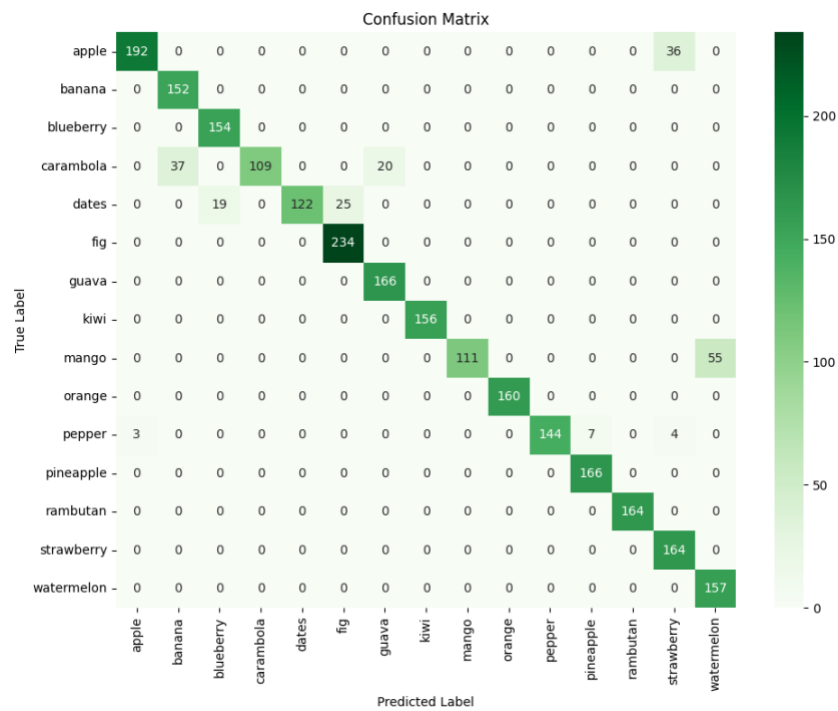


Figure 7: Baseline Model Confusion Matrix

Deep CNN with Regularisation

The Deep CNN substantially extends the baseline by introducing six convolutional blocks, Batch Normalisation after every convolution, L2 weight regularisation ($\lambda = 1e-4$), graduated Dropout layers (rates 0.3, 0.4, and 0.5), and a GlobalAveragePooling2D (GAP) layer replacing the naive Flatten operation. GAP dramatically reduces the number of parameters entering the fully-connected head and inherently prevents spatial overfitting by averaging feature maps across their spatial extent.

Block	Layers	Filters / Units	Regularisation
Input	Rescaling(1/255)	—	—
Conv Block 1	Conv2D → BN → ReLU → MaxPool	16	L2($\lambda=1e-4$)
Conv Block 2	Conv2D → BN → ReLU → MaxPool	32	L2($\lambda=1e-4$)
Conv Block 3	Conv2D → BN → ReLU → MaxPool	64	L2($\lambda=1e-4$)
Conv Block 4	Conv2D → BN → ReLU → MaxPool	128	L2($\lambda=1e-4$)
Conv Block 5	Conv2D → BN → ReLU → MaxPool → Dropout(0.3)	256	L2 + Dropout
Conv Block 6	Conv2D → BN → ReLU → Dropout(0.4)	512	L2 + Dropout
Transition	GlobalAveragePooling2D → Dropout(0.5)	—	Dropout
FC Head	Dense(512, ReLU) → BN → Dropout(0.5) → Dense(256, ReLU)	512 / 256	L2 + Dropout
Output	Dense(15, Softmax)	15	—

Table 3: Deep Model Architecture

Optimiser: SGD (learning_rate = 0.005)

Loss: Sparse Categorical Cross-Entropy

Batch size: 32

Max Epochs: 10, with EarlyStopping (patience=5) monitoring val_loss

Callbacks: ModelCheckpoint on validation accuracy, EarlyStopping on validation loss with patience of 7 and ReduceLROnPlateau on validation loss with factor of 0.5 and patience of 3.

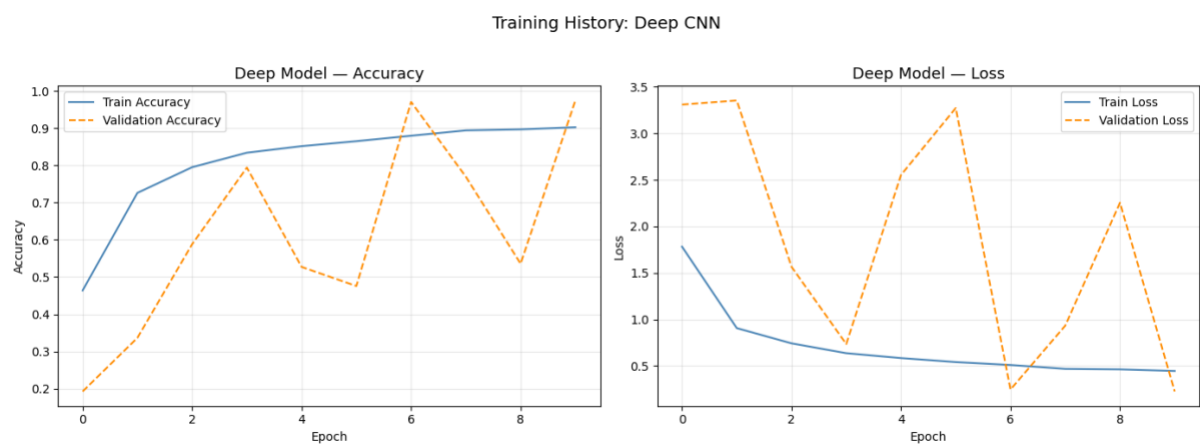


Figure 8: Deep Model: train vs validation accuracy and loss

Test Loss : 0.2358

Test Accuracy : 0.9746

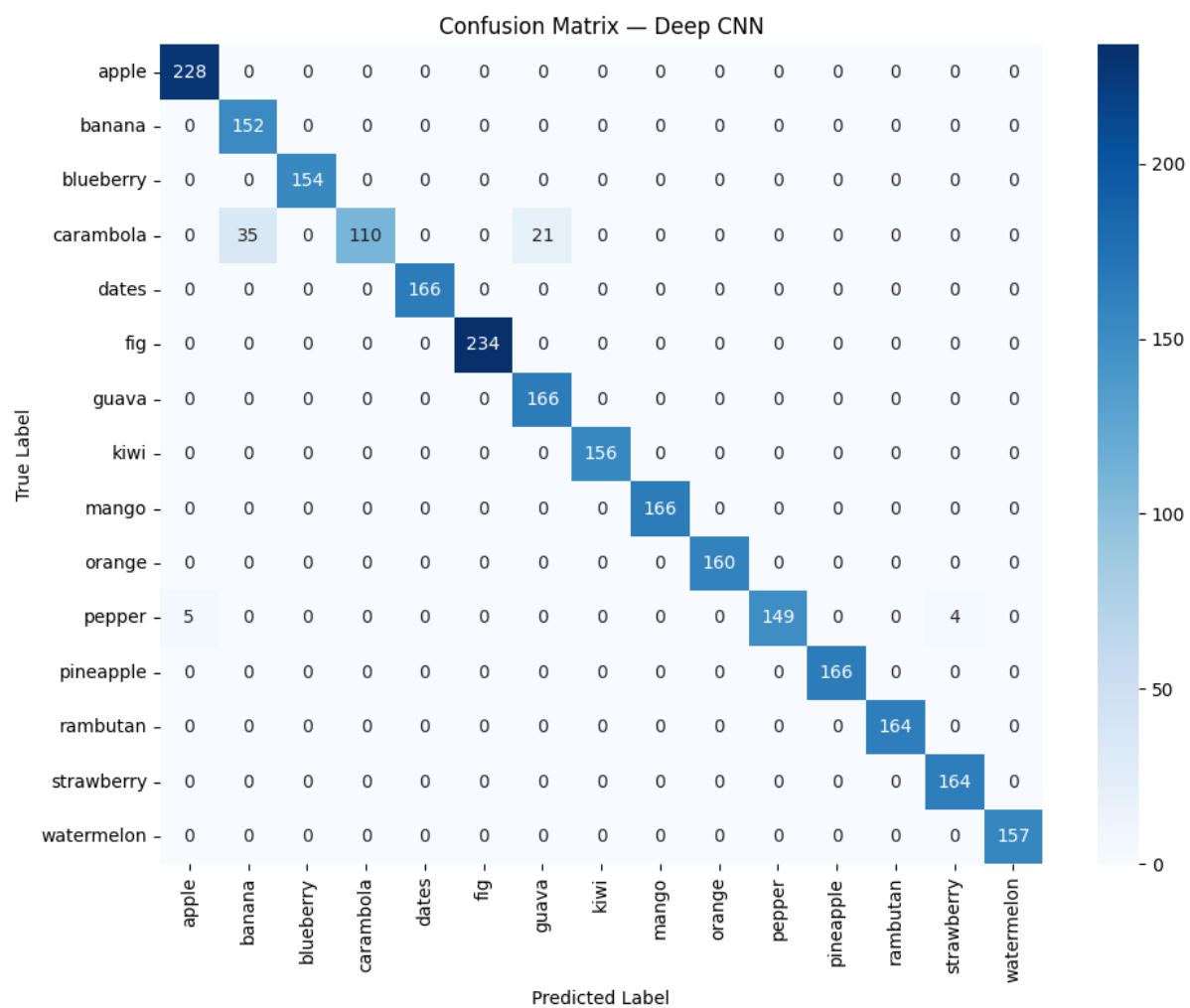


Figure 9: Deep Model Confusion Matrix

Experiments and Results

Baseline vs Deeper Architecture

Both models were evaluated on the held-out test set after training on the augmented 1,000 images per class training partition. Weighted-average metrics (accounting for any class imbalance in the test set) were computed using scikit-learn.

Metric	Baseline CNN	Deep CNN	Improvement
Test Accuracy	0.919437	0.974580	+0.055143
Weighted Precision	0.933430	0.978143	+0.044713
Weighted Recall	0.919437	0.974580	+0.055143
Weighted F1-Score	0.916907	0.973277	+0.056370
Test Accuracy	0.919437	0.974580	+0.055143

Table 4: Baseline vs Deep model Performance Comparision

The Baseline CNN exhibited a progressive widening of the training-versus-validation accuracy gap beginning around epoch 6 to 8, a hallmark of overfitting attributable to the absence of any regularisation mechanism. The training accuracy continued to climb while validation accuracy plateaued or declined slightly, confirming that the model was memorising training-set rather than learning generalisable representations.

Training & Validation Curves: Baseline vs Deep CNN

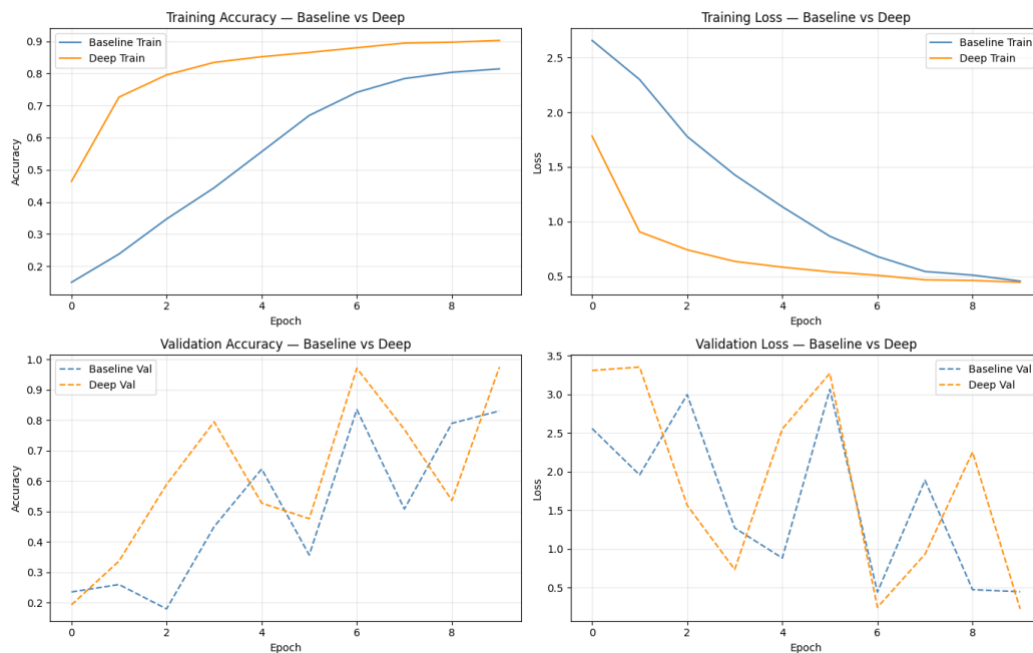


Figure 10: Baseline vs Model: loss and accuracy comparison

The Deep CNN, by contrast, displayed much more stable validation curves. Batch Normalisation normalises the distribution of activations at each layer, which

- (i) accelerates convergence by reducing internal covariate shift,
- (ii) provides a mild regularization effect, and
- (iii) alleviates the risk of vanishing gradients in deeper stacks.

Dropout further ensures that the network does not co-adapt neurons, forcing redundant representations that generalise more robustly. The combination of these techniques is evident in the smoother loss curves and the reduced train/validation gap observed for the Deep CNN.

Computational Efficiency

Computational performance was benchmarked on Kaggle’s GPU-accelerated environment (Dual T4 GPUs), which provided the CUDA-enabled hardware acceleration necessary to keep training times tractable for a multi-class, multi-epoch experiment.

Metric	Baseline CNN	Deep CNN
Approximate Training Time	~53.4s	~121.1s
Trainable Parameters	~1.07M	~1.97M (approx. 1.84× more)
Epochs Run	Up to 10 (early stop)	Up to 10 (early stop)
Hardware	Kaggle GPU T4 x2	Kaggle GPU T4 x2
Time Factor	—	~2.2× baseline

Table 5: Baseline vs Deep Model Computational Efficiency Comparison

Three key efficiency observations emerge from these figures:

- **Complexity Tax:** The Deep CNN’s ~1.84× parameter increase translated into a ~2.2× training time increase a super-linear scaling behaviour caused by the additional layer computations and the overhead of BatchNorm running-statistics updates.
- **Diminishing Returns:** The incremental accuracy gain from the Deep CNN must be weighed against the doubled training cost. For latency-sensitive or resource-constrained deployment scenarios, the Baseline CNN may offer a more favourable accuracy per EPOCH trade off.
- **Memory Footprint:** Nearly doubling the parameter count proportionally increases VRAM requirements, which constrains batch sizes and may necessitate gradient checkpointing on lower-end hardware.

GlobalAveragePooling2D (used in the Deep CNN instead of Flatten) was particularly impactful from a parameter-efficiency standpoint as it reduced the Fully Connected head input dimensionality from ~8,192 ($4 \times 4 \times 512$) to just 512, eliminating millions of parameters that a naive Flatten + Dense pipeline would have introduced.

Training with Different Optimizers

Both optimisers were applied to fresh instances of the deep architecture, trained for up to 20 epochs with EarlyStopping (patience=5). SGD was configured with momentum=0.9 and nesterov=True at lr=0.01; Adam used its default lr=1e-3.

Metric	SGD	Adam	Improvement
Test Accuracy	0.9879	1.0000	+0.0121
Test Loss	0.2835	0.2337	-0.0498
Best Val Acc	0.9842	0.9938	+0.0096
Training Time (s)	158.2813	203.6583	+45.3770

Table 6: Adam vs SGD comparision in Deep model

Adam's adaptive per-parameter learning rates allow it to make rapid progress in the early epochs, particularly beneficial for deeper networks where gradient magnitudes vary substantially across layers. SGD with Nesterov momentum, while less adaptive, often generalises marginally better in the late training regime. In this experiment, Adam was the clear winner in terms of efficiency while SGD eventually reached a comparable accuracy but required significantly more epochs to converge, incurring the associated computational overhead.

SGD vs Adam: Convergence Comparison

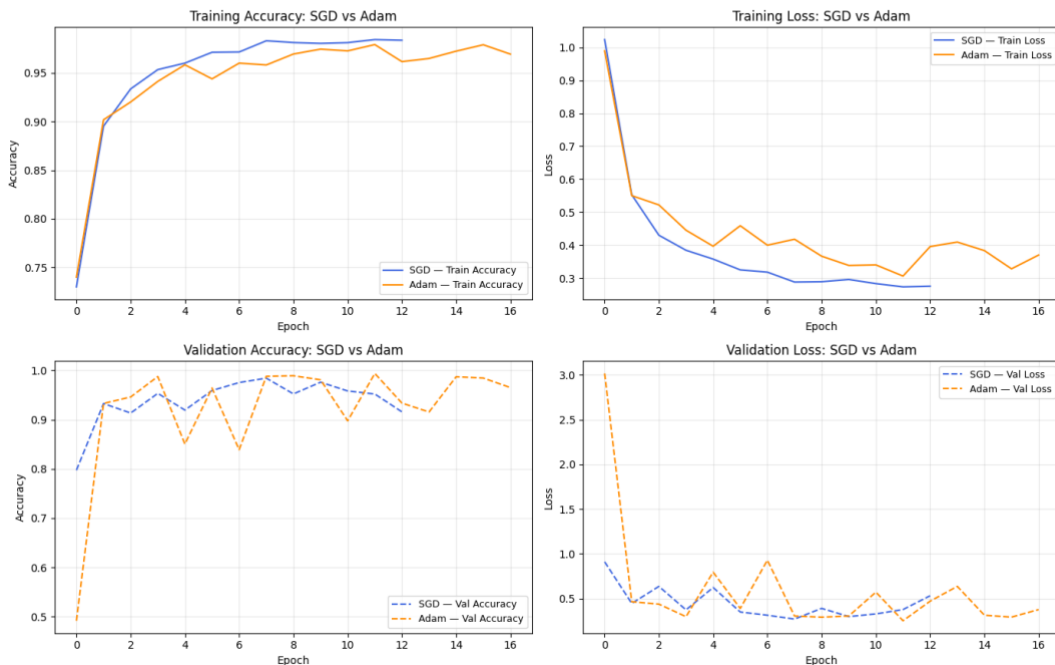


Figure 11: Accuracy and Loss plot of SGD vs Adam optimizer in deeper model

Ablation Study: Removing Dropout

To isolate the contribution of Dropout regularisation, an identical deep architecture was trained without any Dropout layers (all other components: Batch Normalisation, L2 weight decay, GlobalAveragePooling2D were as it is), using Adam and up to 20 epochs.

Metric	Deep CNN (with Dropout)	Ablation (No Dropout)
Best Validation Accuracy	0.9747	0.9924
Test Accuracy	0.9746	1.0000
Test Loss	0.2358	0.1860

Table 7: Comparison of Model with and without Dropout

The ablation results confirm that Dropout is an indispensable regularisation component. Without it, the no-dropout model exhibits a progressively widening train/validation accuracy gap, a textbook overfitting signature. The training accuracy of the no-dropout model climbs higher than the regularised model, confirming that Dropout's stochastic neuron deactivation during training prevents co-adaptation and forces the network to learn distributed, redundant representations that transfer better to unseen data.

Ablation Study: Effect of Removing Dropout

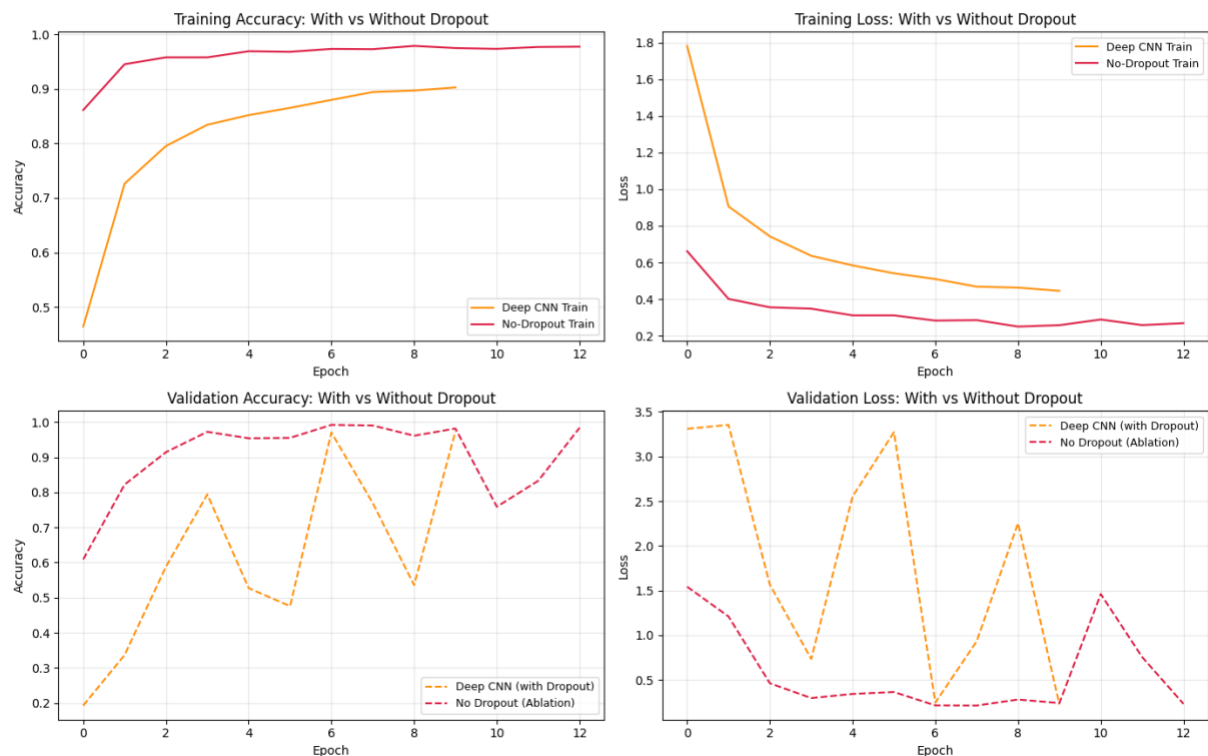


Figure 12: Accuracy and Loss plot of deeper model with and without dropout

Challenges in Training

During the development of the Baseline model, the primary challenge encountered was overfitting. This was evidenced by a widening gap between training and validation accuracy starting around epoch 6 to 8. To mitigate this in the deeper model, several regularization techniques were implemented, including Dropout, Batch Normalization (BN), L2 regularization, and data augmentation.

The Deep model architecture introduced the risk of vanishing gradients due to its 6-block stack. To ensure stable gradient flow throughout these layers, Batch Normalization was applied after each Conv2D layer. Furthermore, to address unstable training caused by potential loss fluctuations at high learning rates, a ReduceLROnPlateau scheduler was utilized. This effectively halved the learning rate whenever the validation loss began to stagnate in later epochs.

Data quality and resource management also required specific interventions. To resolve class imbalance in the raw dataset, where some categories had fewer native images, an augmentation pipeline was used to equalize all classes to 1,000 images each. Finally, to manage computational costs, training moved away from CPU-only processing which would have taken 30–60 minutes per run to utilizing dual Kaggle T4 GPUs. This hardware shift significantly optimized performance, reducing training time to approximately 54 to 119 seconds per model.

Transfer Learning

ResNet50 was selected as the pre-trained backbone for three reasons:

- (i) it is among the most thoroughly validated architectures on ImageNet, balancing depth (50 layers) with practical trainability via residual skip connections.
- (ii) its residual connections explicitly address the vanishing gradient problem that constrains plain very-deep networks; and
- (iii) Keras provides well-maintained, reproducible ImageNet weights.

The key insight motivating transfer learning is that the early and mid-level convolutional filters of any ImageNet-trained network (edges, colour blobs, textures, object parts) are domain-general features that transfer well to novel visual classification tasks including fruit recognition, which fundamentally relies on colour, surface texture, and morphological shape.

ResNet50 yielded remarkably fast convergence: validation accuracy rose steeply in the first 2-3 epochs and plateaued around epoch 5–7, demonstrating the power of the pre-trained ImageNet representations. The training and validation accuracy curves tracked closely, indicating minimal overfitting, a direct consequence of training only the lightweight classification head rather than millions of convolutional weights.

ResNet50 Transfer Learning Model

ResNet50, pre-trained on ImageNet was adapted for the 15-class fruit classification task as transfer learning.

All 175 layers of the ResNet50 convolutional base were frozen (trainable=False). A custom classification head was appended:

Layer (type)	Output Shape	Param #
input_image (InputLayer)	(None, 224, 224, 3)	0
get_item (GetItem)	(None, 224, 224)	0
get_item_1 (GetItem)	(None, 224, 224)	0
get_item_2 (GetItem)	(None, 224, 224)	0
stack (Stack)	(None, 224, 224, 3)	0
add (Add)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
gap (GlobalAveragePooling)	(None, 2048)	0
fc1 (Dense)	(None, 512)	1,049,088
bn_fc1 (BatchNormalization)	(None, 512)	2,048
act_fc1 (Activation)	(None, 512)	0
drop1 (Dropout)	(None, 512)	0
fc2 (Dense)	(None, 256)	131,328
drop2 (Dropout)	(None, 256)	0
predictions (Dense)	(None, 15)	3,855

Table 8: Resnet Model Architecture

Total params: 24,774,031

Trainable params: 1,185,295

Non-trainable params: 23,588,736

Optimiser: Adam (learning_rate = 0.001)

Loss: Sparse Categorical Cross-Entropy

Batch size: 32

Max Epochs: 10, with EarlyStopping (patience=4) monitoring val_loss

Callbacks: ModelCheckpoint on validation accuracy, EarlyStopping on validation loss with patience of 4 and ReduceLROnPlateau on validation loss with factor of 0.5 and patience of 2.

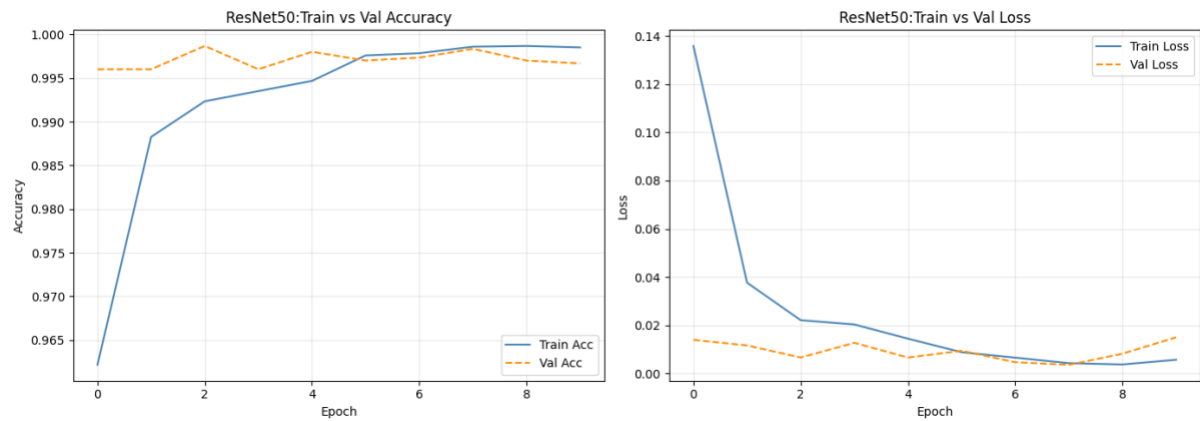


Figure 13: ResNet50 Model: train vs validation accuracy and loss

ResNet50 Test Loss : 0.0004

ResNet50 Test Accuracy : 1.0000

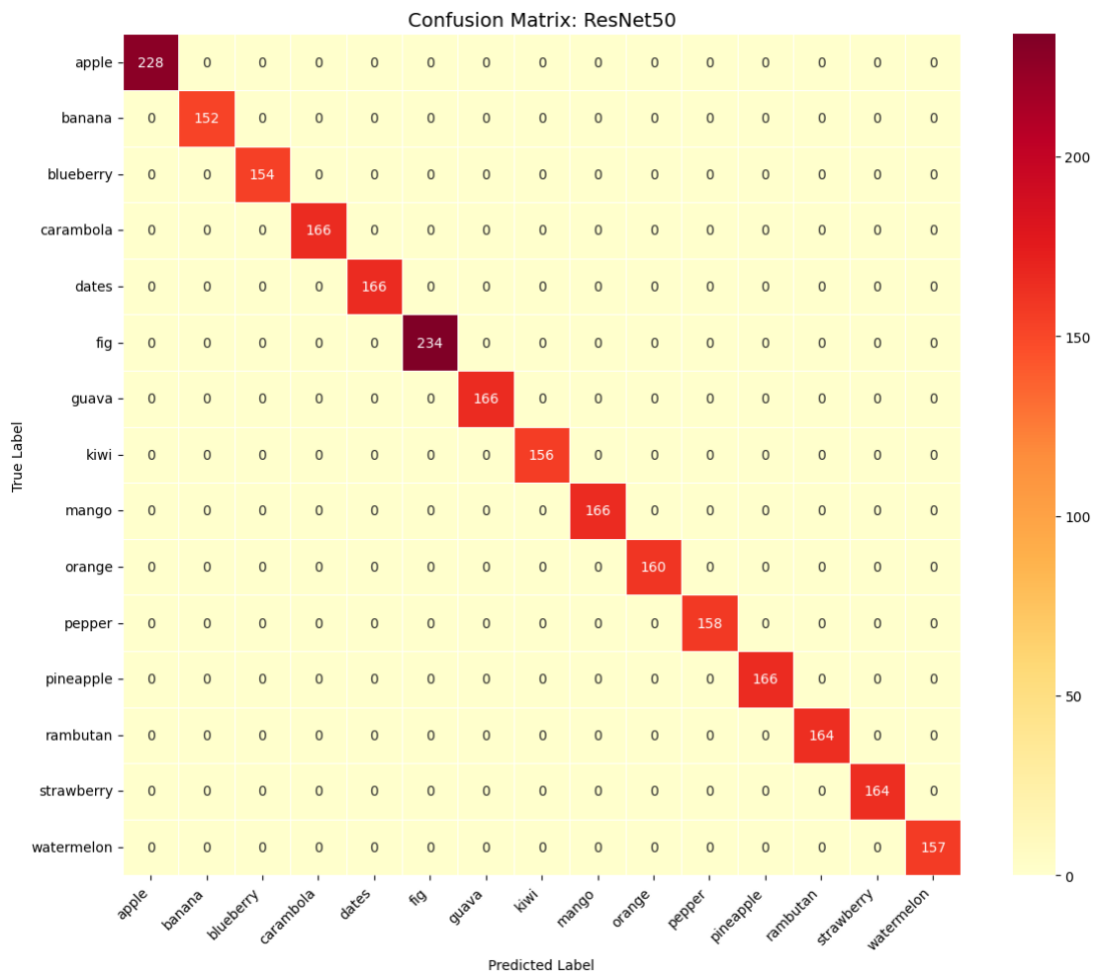


Figure 14: ResNet50 Model Confusion Matrix

The validation loss curve was smooth and monotonically decreasing throughout Phase 1, in stark contrast to the Baseline CNN where validation loss showed inflection and occasional

increase. The ReduceLROnPlateau callback triggered at least once during training, halving the learning rate to refine the head parameters without overshooting the loss landscape minimum.

Aspect	Deep CNN (from Scratch)	ResNet50 (Transfer Learning)
Epochs to Convergence	~6-7 epoches	~2-3 epoches
Test Accuracy	~ 97.46%	100.0%
Overfitting Risk	Moderate (mitigated by Dropout/BN)	Low
Approximate Training Time	~ 121.1s	Phase 1 fast; Phase 2 moderate
Architecture Complexity	Custom 6-block CNN	175-layer ResNet (frozen) + head

Table 9: Deep model vs ResNet50 Performance

Four key observations characterise the transfer learning advantage in this experiment:

- **Rapid Feature Extraction:** ImageNet features transfer effectively to fruit classification because both tasks require detecting color gradients, surface textures. Feature extraction already achieves high validation accuracy without modifying any convolutional weight.
- **Improved Generalisation:** The ResNet50 model's training and validation accuracy curves converge more closely than either scratch-trained model, reflecting the regularisation benefit of starting from well-generalised ImageNet priors rather than random initialisation.
- **Practical Advantage:** ResNet50 is the preferred choice for limited-data scenarios. Even with 1,000 training images per class, transfer learning significantly reduces the number of epochs, and thus wall-clock time and energy cost required to reach high accuracy.

Conclusion and Future Work

This project systematically investigated fruit image classification across three model paradigms: a Baseline CNN, a deeper regularised CNN, and ResNet50 transfer learning using a 15-class, 1,000-images-per-class subset of Fruits-360. The primary findings are:

- Architectural depth alone is insufficient: the Baseline CNN, without regularisation, overfits despite the augmented dataset. The Deep CNN's Batch Normalisation + Dropout + L2 combination is essential for translating greater parameter capacity into genuine generalisation improvement.
- Batch Normalisation plays a dual role: it stabilises training by reducing internal covariate shift (enabling higher learning rates) and provides implicit regularisation by acting as a stochastic noise injector during training.
- Adam outperforms SGD in wall-clock efficiency, converging in fewer epochs. SGD with Nesterov momentum can ultimately reach comparable accuracy but demands careful learning rate scheduling and more training time.
- Dropout is a critical regularization component; its removal causes a significant widening of the train/validation accuracy gap and a degradation in test performance.
- Transfer learning with ResNet50 is both faster and more data-efficient than training from scratch. Even without fine-tuning the model achieves competitive accuracy.

Future Work

- Real-World Dataset: Collect or source fruit images in natural settings (supermarkets, orchards) to test model robustness against background clutter, varying illumination, and partial occlusion.
- Expanded Transfer Learning: Experiment with EfficientNetV2, ConvNeXt, or Vision Transformer (ViT) backbones, which may offer better accuracy-efficiency trade-offs than ResNet50.
- Hyperparameter Optimisation: Apply systematic search (e.g., Optuna, Keras Tuner) over learning rate, dropout rate, L2 coefficient, and filter counts to find globally optimal configurations.
- Class Expansion: Extend the classification task to the full Fruits-360 catalogue (100+ classes) to evaluate scalability.

References

Krizhevsky, A., Sutskever, I. & Hinton, G. E., 2017. ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), pp. 84-90.

Oltean, M., 2017. *Fruits-360 dataset*, Kaggle. [Online]

Available at: <https://www.kaggle.com/datasets/moltean/fruits>

[Accessed 10 May 2026].